

# A Fun Case of Client-Side RCE Plus Server-Side XXE

By J0o1ey · Translated from the Chinese original

## 0x00 Preface

Friends interested in technical exchange or penetration testing training are welcome to contact me via QQ/WeChat: 547006660.

## 0x01 How It Started

A friend was doing a penetration test for a client — a well-known insurance company. The systems were tough as nails, and after half a day nothing had popped.

He called me over to take a look together, and that's how this article came about.

## 0x02 A Client-Side RCE

My friend sent me the target. It was a page with nothing but two download links.



The downloaded files looked like this. The directory contained a bundled JRE, which made it obvious: this client exe was just a wrapper around a Java launch command.



I then opened the program folder and reverse-engineered the Jar inside.

The full\_path variable was earlier defined as the path the user types when installing an update.

```

server_path : String
servlet_url : String
UpdateDialog()
actionPerformed(ActionEvent)
bakDownloadFiles(File) : void
cancel() : void
checkCodeVersion(File, URL)
connect2server() : void
361 String full_path = bat_file.getPath().replace(" ", "\\ ").replace("{", "\\{").replace("}", "\\}");
363 String cmd = "cmd /c " + pre_cmd + " " + full_path;
365 if (new File("program/cmd.exe").exists()) {
366     cmd = new File("program/cmd.exe").getAbsolutePath() + " /c " + pre_cmd + " " + full_path;
369 System.out.println(cmd);
370 Process pro = Runtime.getRuntime().exec(cmd);
}

```

So it's simple: `full_path` is user-controlled. Trick the user into entering an injected command as the installation path:

```
D:\software && ping hacker's IP
```

### 0x03 Discovering a Webservice SOAP Interface

One lucky client-side RCE certainly isn't enough, so I moved on to dig into the server side.

There were no other functional points to speak of, so I simply FUZZed the system's third-level directories.

```

ubuntu@VM-8-6-ubuntu:~/ffuf$ ./ffuf -w content.txt -u http://[redacted]/FUZZ
v1.3.1

:: Method      : GET
:: URL         : http://[redacted]/FUZZ
:: Wordlist     : FUZZ: content.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

```

The fuzzing turned up a webservice interface:

```
http://.xxxxxxx.cn/xxxx/service
```

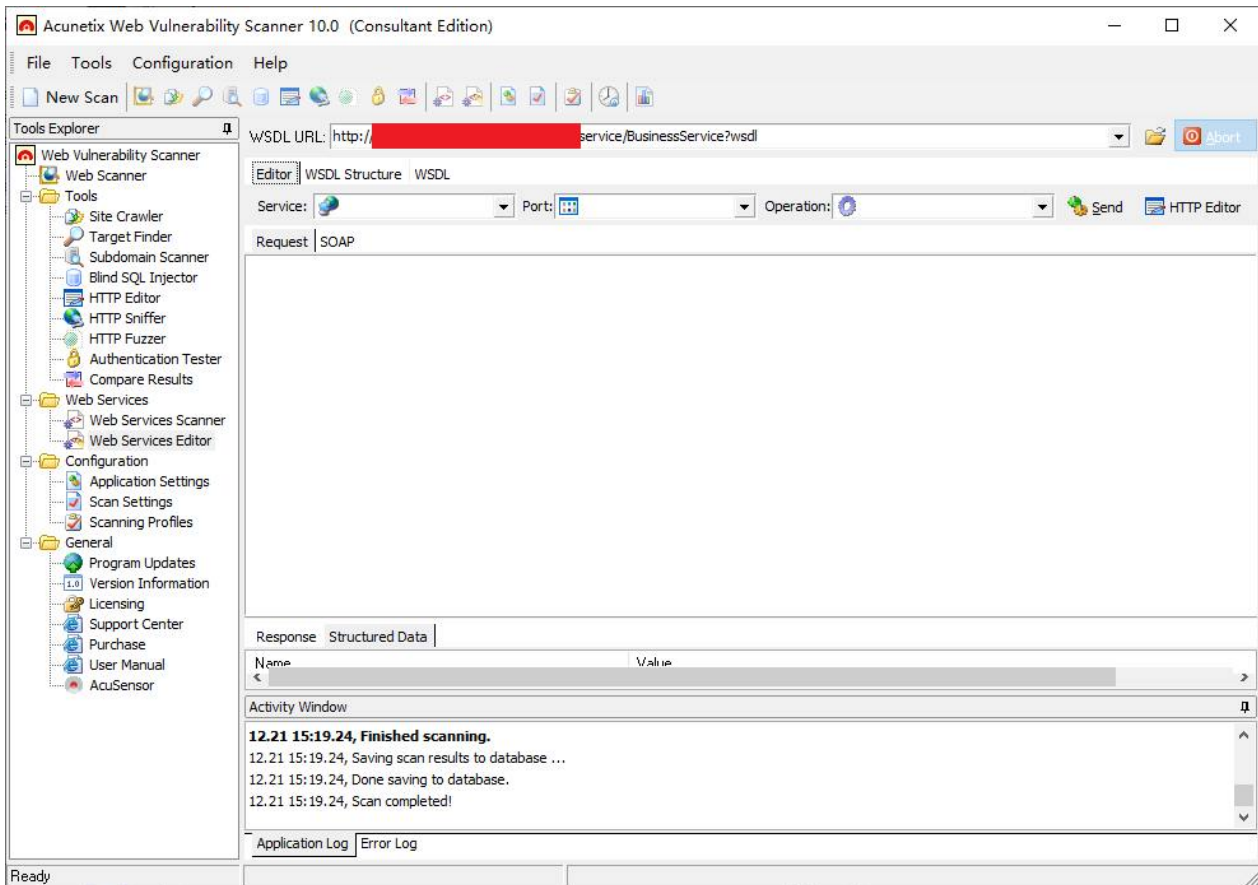
Available SOAP services:

|                                                                               |                                                                                                                                                                |
|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Business</b> <ul style="list-style-type: none"> <li>callService</li> </ul> | <b>Endpoint address:</b> [redacted]acs:[redacted]/service/BusinessService<br><b>WSDL :</b> { [redacted]bs.nc/ }<br><b>Target namespace:</b> http://eprk.bs.nc/ |
|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|

I pieced together its WSDL endpoint:

```
http://.xxxxxxx.cn/xxxx/service/BusinessService?wsdl
```

But when I imported it into the debugging modules of SoapUI or AWVS, the import failed.



I looked carefully at what the WSDL returned... damn. Both the WSDL Import Location and the Soap Address Location were intranet domains.



Fortunately, the same WSDL endpoint was also reachable by appending the path to the system's external domain.

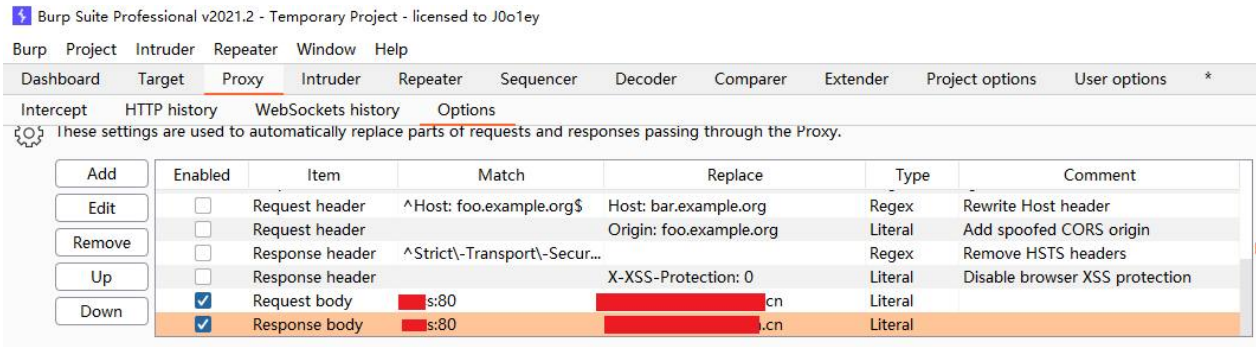
But automated SOAP debugging tools "import whatever they see", which left us stuck.

## 0x04 Importing into SoapUI

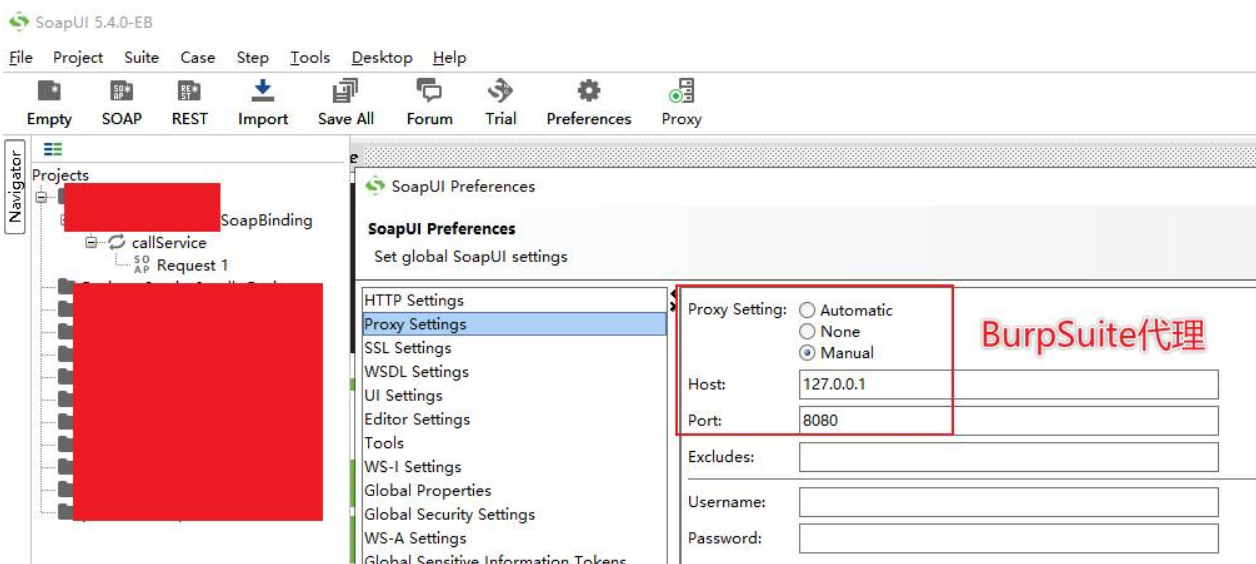
After thinking for a moment, I suddenly remembered that Burp Suite can rewrite Request Body and Response Body values on the fly — heh, that gives us a way to import it.

In Burp Suite's Proxy Options, I added Match & Replace rules:

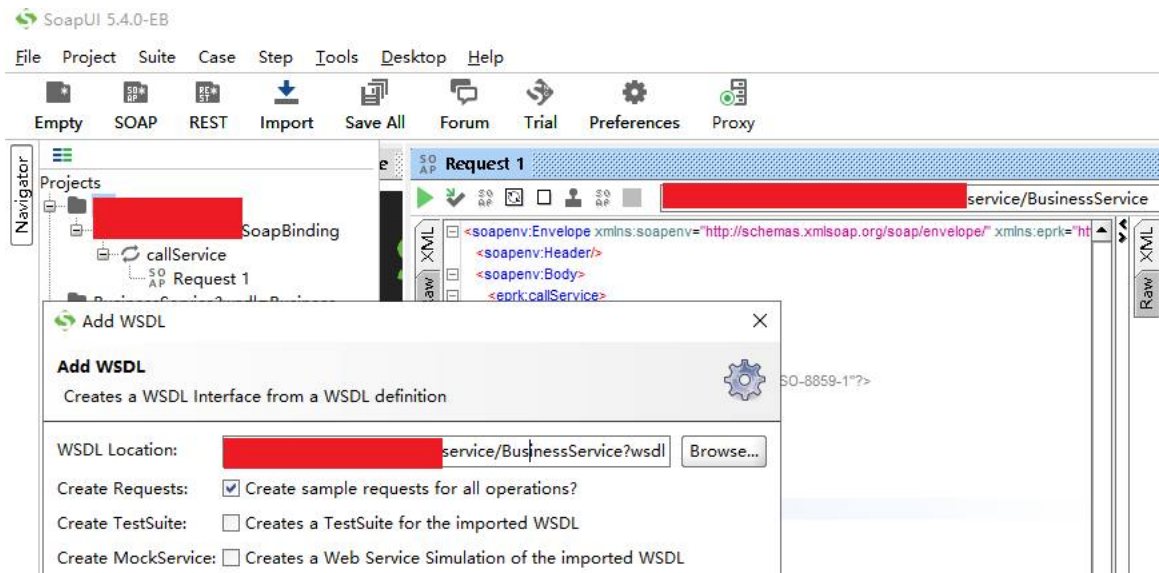
replacing the intranet domains in both the WSDL Import Location and the Soap Address Location with the external domain.



Then I set a proxy in SoapUI:



With the proxy on, I added the WSDL again. The intranet domains in the Response Body were successfully replaced, and the WSDL imported cleanly~

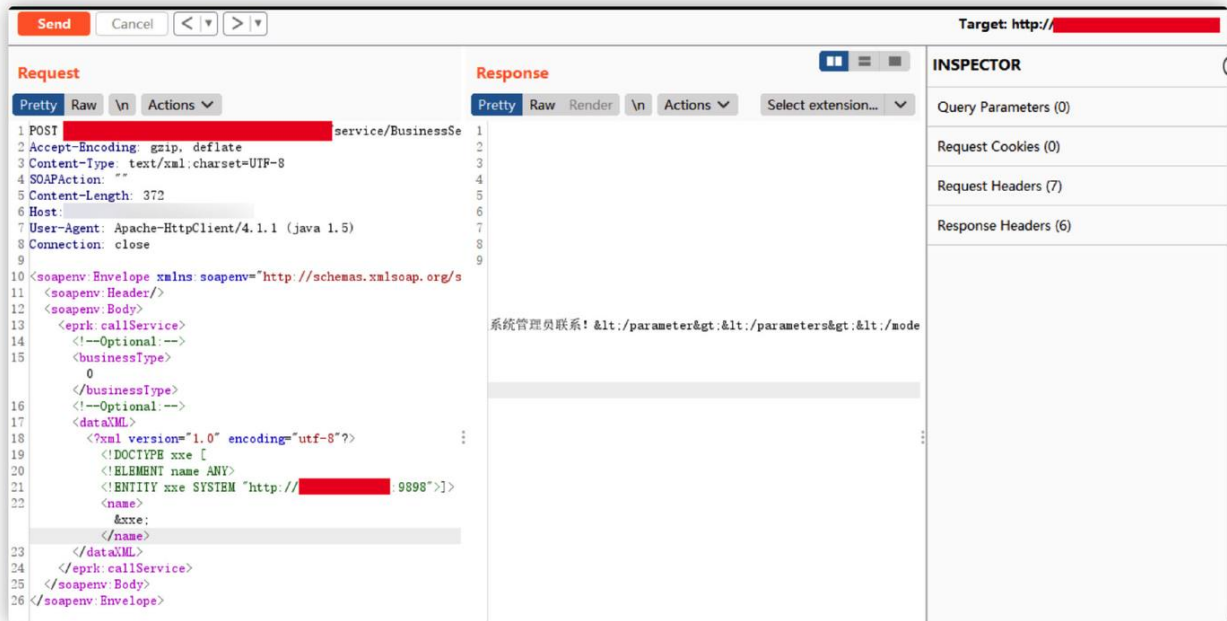


## 0x05 Hunting the XXE

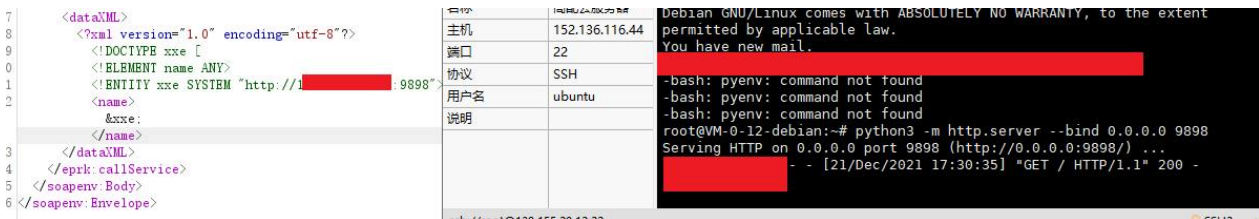
After importing the interfaces, I noticed a parameter named dataXML — I grinned inside; an XXE was practically gift-wrapped.



I went straight to Burp Suite and tested XXE out-of-band.



The OOB callback came through. XXE secured, packing up~



## 0x06 Summary

Hold on just a little longer — after the clouds part, the moon shines through. The bug is right in front of you.